TASK 2 | METHODOLOGY PASSPORT CARD

Copilot Interaction Mode and Productivity Across Programmer Experience Levels

Authors: Makhmutov Damir; Kasimov Amir

Public GitHub repository:
https://github.com/kirillmehopov-design/research-methods-task2-makhmutov-and-kasimov

1. Research type and design

Quantitative secondary analysis of a published quasi-experiment. We use the productivity tables from Guglielmi et al. for 33 participants (17 novices and 16 experts). Each completed two different Java maintenance tasks: one with automatic Copilot suggestions enabled and one with suggestions requested manually. The dataset contains 66 task sessions. Manual invocation is the within-person baseline; Copilot is available in both conditions.

RQ1: How does productivity change between manual and automatic suggestions among novices?
RQ2: How does productivity change between these modes among experts?
RQ3: Does the average mode-related productivity change differ between the experience groups?

2. Testable hypotheses

Let P = completed task fraction per hour and D = $P_{automatic}$ - P_{manual} for each participant. RQ3 is primary. All alternatives are two-sided.

Research question	Null hypothesis (H0)	Alternative hypothesis (H1)
RQ1 Novices	H0_1: Mean D among novices equals zero.	H1_1: Mean D among novices differs from zero.
RQ2 Experts	H0_2: Mean D among experts equals zero.	H1_2: Mean D among experts differs from zero.
RQ3 Group difference	H0_3: Mean D among experts equals mean D among novices.	H1_3: Mean D among experts differs from mean D among novices.

3. Variable matrix

Variable type	Definition and measurement
Factors	Copilot interaction mode: automatic/manual; experience group: novice/expert, using the source classifications.
Primary outcome	Expert mean paired productivity change minus novice mean paired change (completed task fraction/hour).
Secondary outcomes	Within-group productivity changes; task completion (%); session time (minutes).
Design and controls	Task A/B and assignment group A/B/C/D are retained. Source setting: Java maintenance tasks, Copilot, common instructions and scoring. Task/order sensitivity checks are planned.

Experience definitions: novices were computing students with less than three years of programming experience; experts were industry developers with at least three years. Experience is observed, not randomly assigned. We retain the source labels.

4. Primary metric, guardrail and baseline

Productivity: P = (completion percent / 100) / (time minutes / 60). For example, 50% completion in 60 minutes gives 0.5 completed task fraction/hour. We recalculate P from the source values and check it against the published PRODa column.

Primary metric: mean(D | expert) - mean(D | novice), where D = $P_{automatic}$ - P_{manual}. Positive values indicate a larger automatic-mode productivity benefit for experts; negative values indicate a smaller benefit or greater cost relative to novices.

Completion guardrail: within each group, flag an average automatic-minus-manual completion change below -5 percentage points. This is our practical reporting threshold. It is not a validated code-quality or safety standard. Completion and time are also reported separately; shorter sessions do not necessarily mean successful task completion.

Baseline: manually requested Copilot suggestions for the same participant. Each participant performs a different task in the other mode, with task/mode allocation and order defined by the source assignment scheme. The baseline is not an AI-free condition.

5. Data source and analysis plan

We use the two original productivity CSV files from the source replication package. Their bytes are preserved, and SHA-256 checksums are stored in provenance.json. Validation requires finite values, completion from 0 to 100, positive time, consistent labels and exactly one observation per participant and mode. Zero-completion sessions are retained. The eight-row test sample contains real source records from four complete participant pairs.

The current pipeline computes means, medians, p95, paired mean changes and percentile-bootstrap 95% confidence intervals. Bootstrap uses 10,000 repetitions and seed 42, resampling whole participant pairs separately within experience groups. The primary contrast is evaluated directly; significance in one group alone does not establish a group difference.

The current estimates are unadjusted. The next analysis stage will examine task and order effects and leave-one-participant-out sensitivity. Published findings and data were available before this plan, so the study is an exploratory replication. The sample and task scope limit generalization; no causal effect of experience itself is claimed.

6. Reproducible implementation

The repository contains README.md, .gitignore, .dockerignore, Dockerfile, requirements.txt, configs, src, tests, data and an MIT LICENSE. Python 3.12.12 is pinned by version and official image digest. The pipeline uses only the standard library. Twelve unit tests, the Docker build and the full-data run have been verified locally.

Docker quickstart:
docker build -t copilot-mode-study .
docker run --rm copilot-mode-study

Tests run during the build. The executable validates inputs and writes observations.csv, paired_changes.csv, results.json and benchmark.log in outputs/. Configuration controls source paths, bootstrap settings and the completion threshold. Generated outputs are excluded from Git.

7. Source attribution

Guglielmi et al. How do Copilot Suggestions Impact Developers' Frustration and Productivity?
https://doi.org/10.1007/s10664-026-10965-6
Replication package: https://figshare.com/s/2f26b8ee294a3a87c908

The source researchers collected the observations. Our contribution is the secondary analysis and robustness checks. Source data: CC BY 4.0 as stated on Figshare. Our code: MIT.